

Technical Report - RAG-Based Question Answering System

Project Overview

The objective of this project was to design and implement a Retrieval-Augmented Generation (RAG) system capable of answering questions from the AWS Customer Agreement document. The system combines document retrieval, semantic search, large language model inference, SQL-based analytics, and a web dashboard into a single application. The project was developed using FastAPI, Streamlit, SQLite, SQLAlchemy, FAISS, Sentence Transformers, and Ollama. The primary goal was to generate document-grounded answers while reducing hallucinations by providing the language model with retrieved context from the source document.

System Architecture

The system follows a modular architecture consisting of document processing, vector retrieval, language model inference, logging, and analytics components.

Workflow:

PDF Upload → Text Extraction → Chunking → Embedding Generation → FAISS Vector Store

User Question → Question Embedding → Similarity Search → Top-K Retrieval → Retrieved Context → Prompt Construction → Ollama LLM → Answer Generation

Finally, SQLite Logging + Analytics Dashboard

Document Processing and Chunking Strategy

During ingestion, the AWS Customer Agreement PDF is parsed and converted into plain text. The text is then divided into chunks before embedding generation. Chunk Size: 1000 characters Overlap: 100 characters the selected chunk size provides enough context for semantic retrieval while keeping individual chunks small enough for efficient indexing. The overlap helps preserve context. The document used for testing generated dozens of chunks that were subsequently indexed for retrieval.

Embeddings and Vector Search

The embedding model used in this project is all-MiniLM-L6-v2 from Sentence Transformers. This model was selected because:

- Lightweight and fast
- Produces 384-dimensional embeddings
- Suitable for local execution
- Provides strong semantic search performance.

Each chunk is converted into a vector embedding and stored in a FAISS index. When a user asks a question, the same embedding model converts the question into a vector. FAISS then performs similarity search between the question vector and stored document vectors. The Top-K retrieval strategy retrieves the five most relevant chunks, which are used as context for answer generation.

Language Model Integration

Ollama was used to run the language model llama3.2:3b locally. The retrieved chunks are combined with the user question to create a prompt. The language model receives only the retrieved context and is instructed to answer based on the provided information. If the answer cannot be found within the retrieved context, the system returns a message “I could not find the answer in the document”. This helps reduce hallucinated responses.

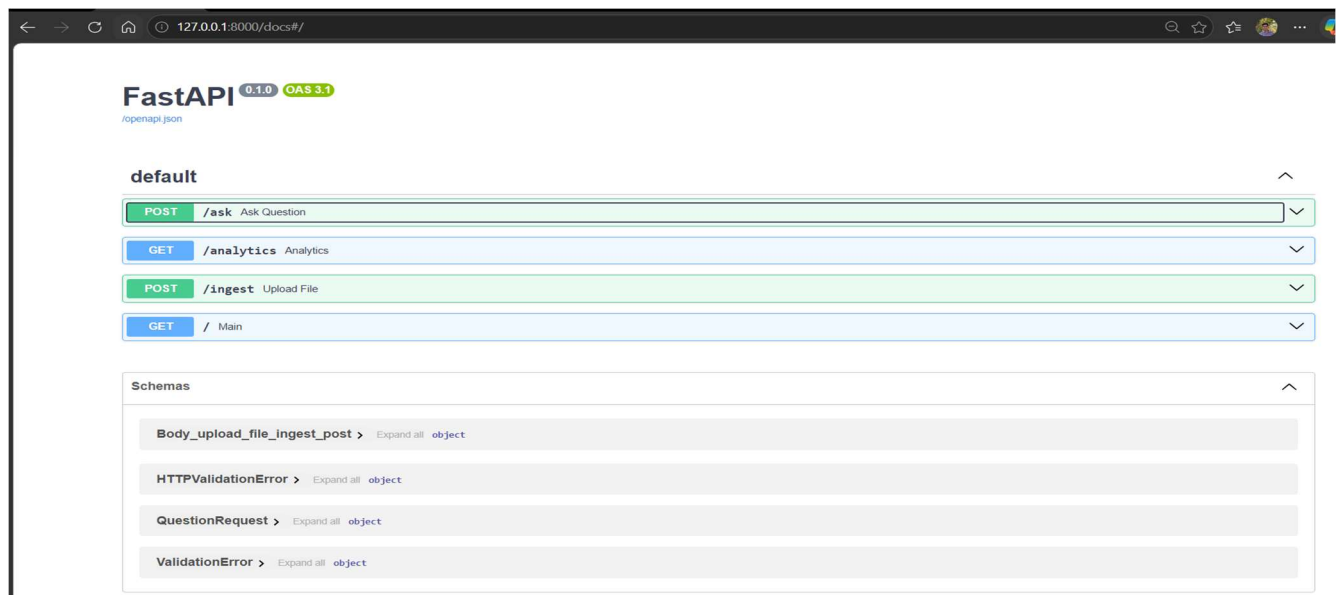
Database Design and Usage Logging

SQLite was selected as the database because it is light weight and file-based database good enough for this task. Every call to the /ask endpoint is logged. Logged fields include:

- User Question
 - Generated Answer
 - Response Time
 - Answer Found Status
 - Timestamp
- These records form the basis of the analytics subsystem.

FastAPI Backend

The backend exposes three primary endpoints: POST /ingest Processes and indexes uploaded pdf documents. POST /ask Accepts a user question, performs retrieval, generates an answer, logs the interaction, and returns answer sources. GET /analytics Returns aggregated analytics data from the SQLite database. Pydantic models are used for request and response validation.

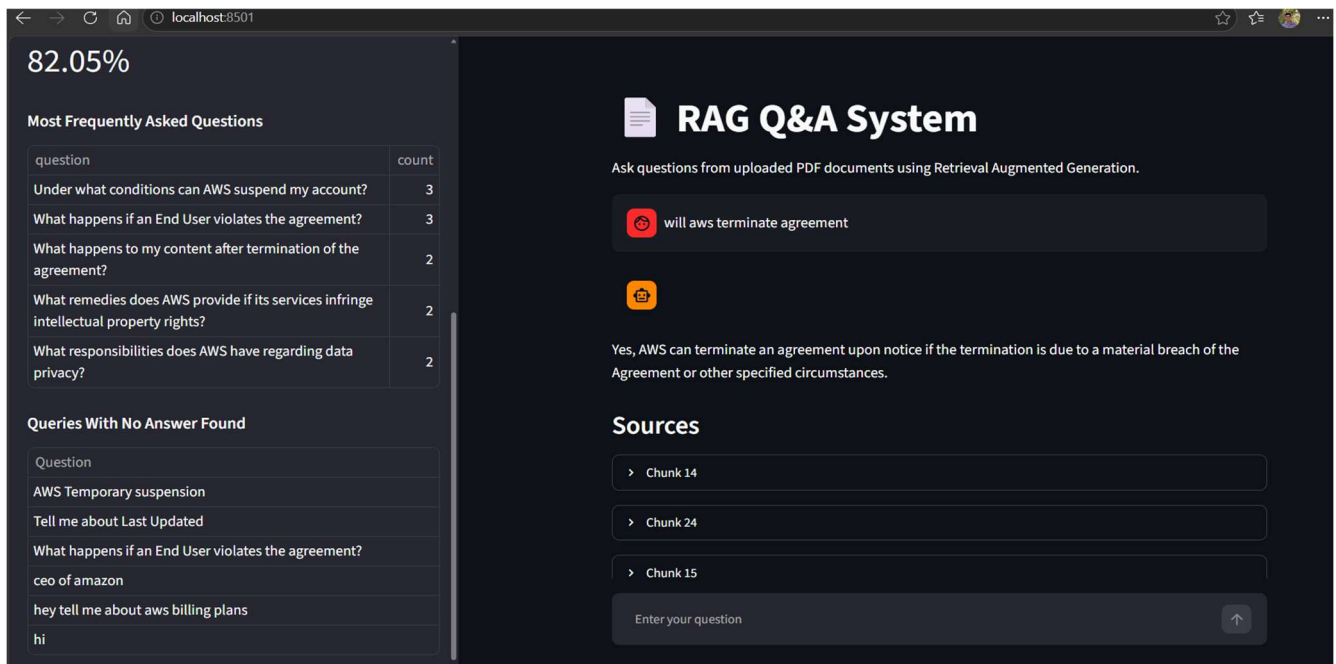
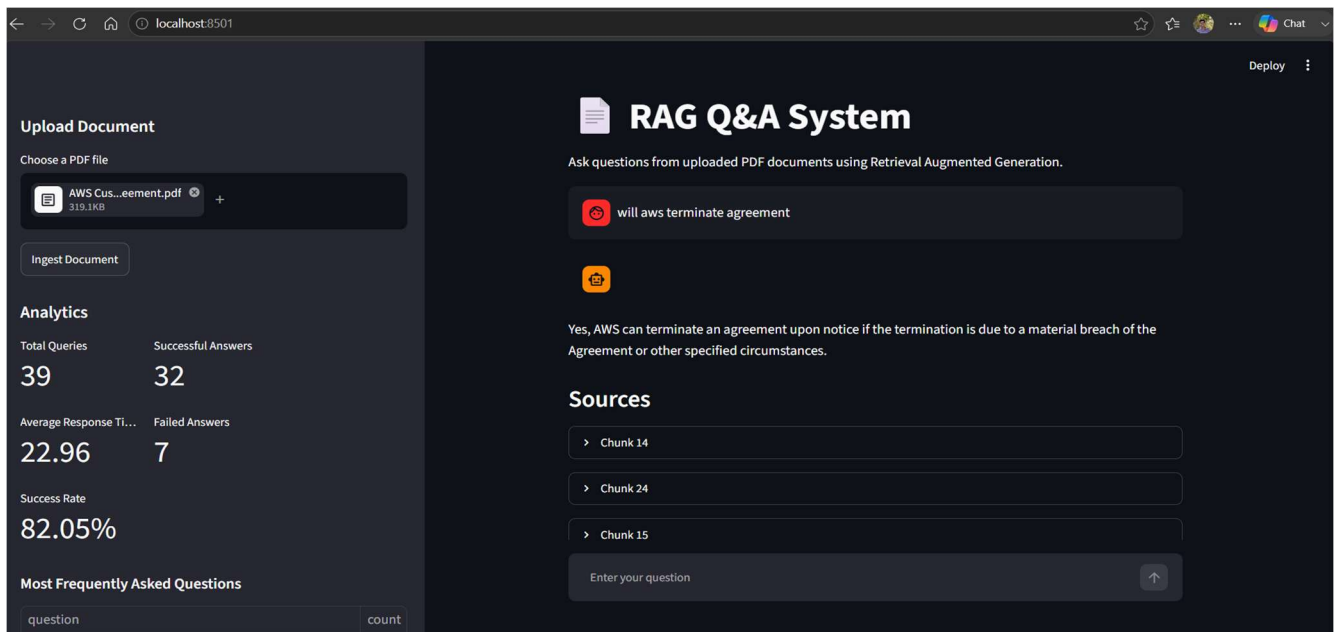


Frontend Dashboard

The frontend was implemented using Streamlit. Features:

- PDF upload interface
- Chat-style question answering
- Source chunk display
- Analytics dashboard

Streamlit communicates with FastAPI using HTTP requests. The frontend and backend run as separate processes.



Error Handling and Edge Cases

Edge cases were handled during development:

- Empty questions
- No document ingested
- Invalid file uploads
- Questions outside the document scope.

Conclusion

This project successfully demonstrates the implementation of a complete Retrieval-Augmented Generation pipeline. The solution combines semantic search, vector databases, local LLM inference, SQL analytics, and a web-based dashboard. The application provides document-based answers, source retrieval, and usage analytics.